REASONABLE-UX · BUILD LOG

Architecture Decisions

What was built, why it was built that way, what broke, and what was learned — 117 commits.

Ryan Walsh · April 2026

reasonable-ux started as a single Playwright + Claude loop that took a screenshot, asked "what do you see?", and clicked the next thing. By the end it was over 100 commits: multi-provider LLM routing, authenticated multi-page crawls, Langfuse observability with a hard-won memory-leak fix, a calibrated eval harness with regression detection, and a PDF reporting pipeline backed by Haiku synthesis. This document records the decisions that shaped the system — what was chosen, what was considered, what was given up, and what broke along the way.

---

1 Token Economics

Vision models are expensive per-call. Multi-step runs with 8–12 screenshots, a JSON schema, and a growing conversation history compound fast. Each eval run came out of pocket — this is a self-funded project. Every decision in this section was made under that constraint.

Strip images from conversation history after each step

CONTEXT

Each agent step appends a new user message to the conversation history, which is re-sent in full on the next call. Screenshots are large base64-encoded blobs. After 8 steps, the history contains 8 screenshots that will be re-sent on every subsequent call.

DECISION

Before appending the current step's screenshot, walk all prior user messages and drop any `"type": "image"` content blocks. Only the new step's screenshot travels in the payload. `agent_core.py:850`

ALTERNATIVES CONSIDERED

Keep full multi-turn vision context so the model can reason across frames; window to the last N screenshots rather than stripping all.

TRADEOFFS ACCEPTED

The model loses visual continuity between steps. This is acceptable because Playwright navigates to a new page state on each step — the prior screenshot doesn't reflect what's currently on screen anyway.

OUTCOME

Enabled 12-step defaults without hitting cost ceilings. Image history was the largest single cost driver; stripping it was the highest-leverage change in the section.

Two JPEG quality tiers — 40 for steps, 60 for full-page

CONTEXT

Per-step screenshots are encoded, sent, then immediately stripped (see above). The full-page below-fold screenshot is a single larger call used once per run.

DECISION

Two constants: quality 40 for per-step screenshots (`agent_core.py:221`), quality 60 for the below-fold full-page capture (`agent_core.py:396`, `agent_core.py:403`).

ALTERNATIVES CONSIDERED

Single quality level everywhere (80); PNG for lossless fidelity; adaptive quality based on image content.

TRADEOFFS ACCEPTED

Slightly lower visual fidelity on per-step screenshots. In practice, buttons, nav labels, and body copy remain legible at quality 40. The full-page screenshot warrants 60 because it covers more visual surface and is used for a single more detailed analysis call.

OUTCOME

Materially smaller payloads on per-step screenshots without legibility loss. No fidelity regressions observed across author-run evaluations. No systematic test has been conducted at alternative quality levels — the numbers were set once during early development and have not been revisited.

Model tiering — Sonnet executor, Haiku synthesizer

CONTEXT

Early eval runs used Opus 4.5 as the default executor. A single eval run cost approximately $8. That's not viable for iterative calibration work.

DECISION

Swapped the default executor from Opus to `claude-sonnet-4-6` (Batch 31). Reserved `claude-haiku-4-5-20251001` for the exec summary synthesis step, which has lower quality requirements (`generate_report.py:91`).

ALTERNATIVES CONSIDERED

Opus everywhere for maximum quality; Haiku everywhere for minimum cost; a cost gate that escalates to Opus when confidence is low.

TRADEOFFS ACCEPTED

Sonnet emitted malformed JSON on a substantial share of steps initially — a smoke test fired the `json-repair` fallback on 3 of 7 steps (~40%); the first full eval after the swap fired repair on 49 of 58 steps (84.5%). This required adding the fallback (Batch 32) and a full baseline recalibration (Batch 33). The quality gap is real but acceptable for a tool that surfaces directional UX signals, not precise scores.

OUTCOME

Cost down ~4× ($8 Opus → ~$2 Sonnet per 20-URL eval run, sourced from the Batch 31 Anthropic-dashboard reconciliation). The swap regressed the pass rate to 5/20 against Opus-tuned bands; Batch 33 recalibrated labels for Sonnet's actual scoring behavior, restoring 19/20. Haiku exec synthesis runs at an estimated ~$0.002 per report.

Check token budget after the LLM call, not before

CONTEXT

Without a hard cap, long multi-page runs could accumulate unbounded token spend. But observability requires that each call's data land in Langfuse before the process exits.

DECISION

`step_budget = 2048 if advisor else 1024`. The budget check fires after the call is logged, raising an exception that exits the loop gracefully. `agent_core.py:878`

ALTERNATIVES CONSIDERED

Check budget before each call and refuse if exceeded; let the model manage its own budget via the token-counting API.

TRADEOFFS ACCEPTED

One additional LLM call fires at the budget boundary before the exception is raised.

OUTCOME

Langfuse captures the final step before exit. The report shows "stopped: budget exceeded" rather than a silent timeout. The hard cap has never been the wrong call — runs that hit it were exploratory runs that needed a ceiling.

2 Agent Architecture

LLMAdapter with a special-case path for the Advisor tool

CONTEXT

The tool needs to support Anthropic, OpenAI, and Google models interchangeably. But the advisor-beta tool — which uses Anthropic's extended thinking and tool_use features — is Anthropic-only. LiteLLM doesn't support the advisor-beta API surface.

DECISION

`LLMAdapter.complete()` routes everything through `litellm.acompletion()` (`agent_core.py:154`) except when advisor mode is active, which routes directly to `_complete_anthropic_advisor()` (`agent_core.py:168`). The adapter class unifies tracing across both paths. `agent_core.py:123`

ALTERNATIVES CONSIDERED

Separate provider classes with a factory; hard-code Anthropic everywhere and treat multi-provider as a future concern; run the advisor through LiteLLM with a custom plugin.

TRADEOFFS ACCEPTED

Two code paths to maintain. When `--advisor` is enabled with a non-Anthropic provider, the advisor tool is silently dropped — the request still routes through LiteLLM with the chosen provider, but no advisor capability is available. There is no automatic fallback to Anthropic.

OUTCOME

`--provider openai/gpt-4o` works for standard evaluation runs. Advisor functionality is unaffected by provider choice. The two paths share the same Langfuse tracing hooks.

`nav:<Label>` prefix instead of CSS selectors for navigation

CONTEXT

The agent was emitting CSS selectors like `a[href="/pricing"]` or `a:has-text("Pricing")` for main navigation links. These are fragile: class names change, `:has-text()` is unsupported in Playwright's strict selector engine, and generated markup varies wildly across frameworks.

DECISION

The prompt instructs Claude to emit `nav:` (e.g., `nav:Pricing`) for main-navigation links. The agent loop detects the prefix and routes to `_click_nav_by_label()`, which uses Playwright's semantic `page.get_by_role("link", name=…)` API, with an `a:has-text()` fallback. `agent_core.py:241`

ALTERNATIVES CONSIDERED

CSS selectors everywhere; XPath; Playwright's `get_by_text`; require the user to supply selectors.

TRADEOFFS ACCEPTED

This is a prompt-engineering contract, not a code contract. If the model drifts back to CSS selectors for nav links, clicks fail without an obvious error message. That made regression detection essential.

OUTCOME

Navigation reliability improved substantially across diverse frameworks. The eval harness was extended with `_NAV_DRIFT_RE` (Batch 43) to catch drift automatically — see Eval Harness section below.

Infer persona on step 1, thread it through the entire run

CONTEXT

UX evaluation should reflect a specific buyer archetype, not a generic "user." But requiring a `--persona` flag on every invocation adds friction and produces worse output when users make poor guesses about their own customers.

DECISION

On step 1, `_build_prompt` receives `persona=None` and asks Claude to infer a plausible evaluator persona from the first screenshot, URL, and page title, returning it as a top-level `persona` field in the JSON response. From step 2 onward, the inferred persona is passed back into `_build_prompt` and Claude is told to stay in character. The same string is threaded into the below-fold analysis call. `agent_core.py:294`, `agent_core.py:836`, `agent_core.py:1037`

ALTERNATIVES CONSIDERED

Always require `--persona` flag; re-infer each step; skip personas and evaluate from a neutral perspective.

TRADEOFFS ACCEPTED

The persona is locked after step 1. If the inferred persona is wrong, the entire run reflects it. There is no mid-run correction.

OUTCOME

Multi-site validation (Batch 44) produced site-appropriate personas without user input — DTC sites got "eco-conscious millennial shopper," content platforms got "aspiring paid newsletter creator." The persona is written to `report.json` so runs are reproducible.

---

3 Auth and Multi-Page Orchestration

Pre-authenticate once, serialize browser storage state, reuse across pages

CONTEXT

Multi-page runs against authenticated SaaS products initially required re-authenticating for each page. That meant 3× the runtime cost and a constant risk of session invalidation mid-suite.

DECISION

`_do_auth()` (`run.py:124`) logs in once via a dedicated Playwright context, calls `context.storage_state()` (cookies + localStorage), writes the result to a tempfile, and passes the tempfile path to every subsequent page evaluation via the agent's `storage_state=` parameter. `run.py:227`

ALTERNATIVES CONSIDERED

Re-authenticate per page; pass session tokens as environment variables; API-key-based auth bypass for supported apps.

TRADEOFFS ACCEPTED

Tempfile cleanup is required on every exit path, including error paths. Session expiry between pages is a silent failure mode.

OUTCOME

Multi-step login flows (email → Continue → password → Submit) work reliably. Auth happens once per suite. The auth debug screenshot written to `runs/auth_debug_{PID}.png` on failure (Batch 40) has been the primary diagnostic tool for auth regressions.

Hard stop on auth failure, no silent fallback

CONTEXT
If auth failed silently, the agent would start evaluating the login page as if it were the product — producing confident, detailed, completely worthless UX reports with no indication that anything was wrong.

DECISION
Auth failure raises an exception immediately (Batch 12b). A debug screenshot is written to `runs/auth_debug_{PID}.png` before the exception propagates so there's something to inspect.

ALTERNATIVES CONSIDERED
Log a warning and continue unauthenticated; retry once with a longer timeout; degrade to a single-page unauthenticated run.

TRADEOFFS ACCEPTED
Transient auth failures abort the entire run. There is no automatic retry.

OUTCOME
Forces the caller to fix auth before spending tokens. Every instance of auth failure in practice has been a genuine problem (wrong selector, changed login flow, rate-limited endpoint) rather than a transient blip.

Bounded concurrent page execution (Semaphore 2)

CONTEXT
Sequential execution was the original design (reversed in Batch 62). Wall-clock time for multi-page suites is meaningful when running 5–10 pages; halving it matters.

DECISION
`run_pages()` runs at most 2 pages concurrently via `asyncio.Semaphore(2)`. Pages are launched together via `asyncio.gather` with a configurable stagger delay (`--page-stagger`, default 5s) between start times. `run.py`

RATE LIMIT NOTE
Tier 1 Anthropic limits (50 RPM, 30k ITPM). 2 concurrent pages is borderline at scale but acceptable for typical 4–6 page suites where each page idles between steps (screenshot capture, Playwright navigation) and doesn't saturate ITPM continuously.

FOLDER ATTRIBUTION
`_make_run_dir` was updated to second-precision timestamps (`%H%M%S`) to prevent same-minute folder collision for concurrent same-domain pages. Each concurrent task records `start_time` inside the semaphore immediately before calling `agent_run()`, then `_run_folder_for()` returns the earliest run folder created at or after `start_time - 2s`. Reliable for `--page-stagger ≥ 3`.

PER-PAGE ERROR ISOLATION
`asyncio.gather(..., return_exceptions=True)` means one page's failure does not abort others. Auth tempfile cleanup is still guaranteed via `finally`.

TRADEOFFS ACCEPTED
Stagger < 3s may misattribute folders (documented, not guarded). Rate limit risk increases relative to sequential. Print output from concurrent pages interleaves — acceptable for a CLI tool. Shared auth session is read-only across concurrent contexts; no cross-context state collision.

---

4 Observability — The Langfuse Journey

This section has more narrative than the others because the observability work was the most instructive failure sequence in the project.

Replace `AnthropicInstrumentor` with `@observe` on the three direct-SDK paths

CONTEXT (BATCH 38.1)

LiteLLM's per-step calls were traced via `litellm.callbacks = ["langfuse_otel"]`. But the advisor, below-fold analysis, and scout functions used the Anthropic SDK directly — bypassing LiteLLM entirely. Those three paths accounted for 15–20% of all tokens and were completely invisible in Langfuse.

FIRST ATTEMPT (BATCH 39)

Added `opentelemetry-instrumentation-anthropic` and called `AnthropicInstrumentor().instrument()` after the Langfuse TracerProvider was initialized. Appeared to work. Shipped.

ROOT CAUSE DISCOVERED (BATCH 45)

`AnthropicInstrumentor().instrument()` runs at import time — before the TracerProvider is initialized. The spans were silently discarded. No error. No warning. Just missing traces in the Langfuse UI. The only way to catch this was to count spans per session and notice the gap.

DECISION (BATCH 45)

Removed the entire OTel approach. Replaced with langfuse v4's `@observe(as_type="generation", capture_input=False, capture_output=False)` via the `@_lf_observe` wrapper (`agent_core.py:42`). Each of the three functions (`_complete_anthropic_advisor` at `agent_core.py:167`, `_run_below_fold_analysis` at `agent_core.py:388`, `scout_page` at `agent_core.py:448`) is decorated with `@_lf_observe`. Each call site wraps the invocation in `with propagate_attributes(session_id=run_dir)` to group spans under the correct Langfuse session.

ALTERNATIVES CONSIDERED

Fix the OTel init ordering (fragile — depends on import order); instrument at the `httpx` layer (too low-level, loses semantic context); accept the blind spots.

TRADEOFFS ACCEPTED

Manual `_lf_update_generation()` call required inside each decorated function to preserve prompt/response visibility without triggering auto-capture. More boilerplate per function.

OUTCOME

All three paths surface as named generations in Langfuse under the correct session. Runtime-verified end-to-end via two smoke tests against a live Langfuse instance: a 4-step LiteLLM run produced step generations plus a `_run_below_fold_analysis` generation; a separate `--scout --advisor` run produced `scout_page`, `_complete_anthropic_advisor`, and `_run_below_fold_analysis` generations. Both runs were correctly grouped under their respective `session_id` (`runs//_single_page`), confirming `propagate_attributes` works across both LiteLLM and direct-SDK paths.

`capture_input=False, capture_output=False` — never let `@observe` auto-serialize

CONTEXT

The first version of `@_lf_observe` used default `@observe` settings, which auto-serialize all function arguments and return values to JSON for the trace.

WHAT BROKE

`_run_below_fold_analysis(page, ...)` receives a Playwright `Page` object as its first argument. `@observe` attempted to JSON-serialize it, recursively touching every internal browser and DOM attribute. CPU pegged at 100%. The process consumed over 50 GB of RAM trying to serialize the object graph and had to be killed manually. The same problem would have occurred with `messages` in `_complete_anthropic_advisor` — base64-encoded screenshots inside message content blocks are enormous.

DECISION

`capture_input=False, capture_output=False` on every `@_lf_observe` call (`agent_core.py:52`). Each decorated function manually calls `_lf_update_generation()` after its LLM call, passing only text and scalar values. The advisor's path extracts `safe_input` by stripping image content blocks from messages before logging.

TRADEOFFS ACCEPTED

Prompt/response visibility in Langfuse requires a manual call inside each function. Forgetting it produces a trace with no content.

OUTCOME

Zero memory regression. Trace content is present and readable in Langfuse. This invariant is documented in CLAUDE.md section 5 to prevent future regression.

`propagate_attributes` is a sync context manager — never `async with`

CONTEXT

`propagate_attributes(session_id=...)` returns `_AgnosticContextManager` from `opentelemetry.util._decorator`. It implements `__enter__` and `__exit__` but not `__aenter__` and `__aexit__`.

WHAT BROKE

`async with propagate_attributes(...)` raises a runtime error immediately. The agent loop is async throughout, so this was a natural mistake.

DECISION

Always `with propagate_attributes(...)` (sync context manager). Inside the `with` block, `await` async functions normally — OTel context propagates through `contextvars` across `await` boundaries.

TRADEOFFS ACCEPTED

Slightly counterintuitive in an async codebase. The sync `with` block can contain `await` expressions, which looks wrong but is correct.

OUTCOME

Documented as a section-5 invariant in CLAUDE.md. Has not regressed since.

---

5 Reporting Pipeline

Jinja2 HTML → Playwright headless PDF

CONTEXT

Reports need complex CSS layouts: dark theme, gradient backgrounds, screenshot embeds, per-step tables, score callouts.

DECISION

Render the report to a Jinja2 HTML template written to a temp file, then open it in a headless Playwright Chromium context and call `page.pdf()`. Screenshots are rewritten to `file://` URIs so Chromium can load them locally. `generate_report.py:168`

ALTERNATIVES CONSIDERED

`reportlab` (Python-native but requires manual layout math); `weasyprint` (HTML→PDF without a browser but CSS support is limited); plain text output with no PDF.

TRADEOFFS ACCEPTED

Requires a Playwright/Chromium install. Slower than native PDF generation. The temp HTML file must be cleaned up on every exit path.

OUTCOME

Full HTML/CSS fidelity. The template is editable by anyone who knows HTML. Screenshot embeds render correctly. `document.fonts.ready` is awaited before rendering to ensure fonts load.

Haiku for exec summary synthesis, not deterministic aggregation

CONTEXT

Multi-page runs produce per-page JSON friction lists. A synthesized cross-page summary is more useful than a flat concatenation, but deterministic aggregation (pick the top N friction points by frequency) loses nuance.

DECISION

Pass all page findings to `claude-haiku-4-5-20251001` with a prompt that explicitly requires actionable recommendations: "name actual UI elements, not generic advice." Strict JSON output. Degrades gracefully if Haiku fails — the page-by-page report is always generated regardless. `generate_report.py:61`, `generate_report.py:91`

ALTERNATIVES CONSIDERED

Frequency-weighted top-N aggregation; Sonnet for better synthesis; skip exec summary for single-page runs.

TRADEOFFS ACCEPTED

Non-deterministic. Haiku occasionally produces malformed JSON (~2% of runs). The fallback is an empty exec summary, not a crashed report.

OUTCOME

Synthesized summaries naturally weight findings that appear across multiple pages. Haiku is fast (~2 seconds) and cheap (~$0.002 per report). The strict JSON output constraint has held.

Dedup pass before Haiku synthesis

CONTEXT

If every page in a 4-page run flags "CTA is unclear," Haiku receives and likely repeats that finding four times, wasting space and obscuring other patterns.

DECISION

A 7-line pass in `stitch_reports` iterates page summaries before passing them to Haiku, clearing any `top_finding` string that has already appeared. First occurrence wins. `generate_report.py:287`

ALTERNATIVES CONSIDERED

Let Haiku deduplicate (it sometimes does, unreliably); include all findings and post-process the summary; deduplicate after synthesis.

TRADEOFFS ACCEPTED

Order-dependent — the first page's version of a repeated finding survives. Pages that appear later in the suite may have a better articulation of the same finding.

OUTCOME

Exec summaries present novel cross-page findings. The dedup pass has caught repeated findings in multi-page runs against real SaaS products where a broken nav pattern propagated across every page.

---

6 Eval Harness

Isolated `eval_runs/` directory with a `manifest.json` per run

CONTEXT

Eval runs generate agent artifacts (screenshots, JSON, PDFs) identical in structure to real audit runs. If they land in `runs/`, they pollute the audit history and make it hard to diff eval results over time.

DECISION

Eval output goes to `eval_runs/_/`. Each eval invocation writes `manifest.json` capturing pass rate, per-URL results, labels file SHA, and settings at time of run. `evals/run_evals.py`

ALTERNATIVES CONSIDERED

Tag eval runs in `runs/` with a metadata field; a separate eval database; a dedicated eval repo.

TRADEOFFS ACCEPTED

Two output directories to maintain. `eval_runs/` requires its own cleanup policy.

OUTCOME

Eval manifests are diffable. The same labels SHA means two manifests are directly comparable for regression detection. Audit history in `runs/` stays clean and readable. See `eval_results_sample.md` for a committed sample run.

`_NAV_DRIFT_RE` — automated regression detection for nav selector drift

CONTEXT

Prompt changes and model updates occasionally caused agents to revert to emitting CSS selectors (`a[href="/pricing"]`) for navigation links instead of the required `nav:Pricing` prefix. This produced silent click failures — the selector would fail without an obvious error.

DECISION

A compiled regex pattern at `evals/run_evals.py:137` matches CSS anti-patterns (`a[href`, `a:has-text`, `a:contains`, `.nav-`, `.nav_`, `nav a`, `header a`) in step `target` fields. Labels with `assert_nav_drift: true` fail the eval if the pattern matches any step. All 7 `saas_landing` labels carry this flag. `evals/run_evals.py:143`, `evals/run_evals.py:194`

ALTERNATIVES CONSIDERED

Manual review of eval output after each run; a separate linting pass; no regression detection.

TRADEOFFS ACCEPTED

The regex can false-positive on legitimately complex CSS selectors for non-navigation elements on unusual pages.

OUTCOME

Drift caught automatically on every eval run. The regression that prompted this change (Batch 43) would have been invisible in production without it.

Calibrate eval baseline to the model's actual behavior, not ideal output

CONTEXT

The first locked baseline had a 52.6% pass rate — completely unusable as a regression signal. Running the eval against changes would produce noise rather than signal.

ROOT CAUSE

10 label bugs surfaced iteratively: substring mismatches (`subscribe` matched `subscription`), wrong expected keywords for specific pages, score bands set too narrow around ideal expected values rather than around observed model behavior.

DECISION (BATCHES 28 + 28.5 — OPUS ARC)

Batch 28 fixed 6 label bugs and widened score bands by ±12 (52.6% → 65%). Batch 28.5 fixed 4 more label bugs and recalibrated bands against observed Opus output (65% → 19/20).

DECISION (BATCH 33 — SONNET ARC)

The Opus → Sonnet swap (Batch 31) regressed the pass rate to 5/20 against Opus-tuned bands. Batch 33 recalibrated labels for Sonnet's actual scoring behavior, restoring 19/20 (95%) — the locked baseline used today.

LESSON LEARNED

Narrower score bands are not stricter — they are unstable. A band calibrated to ideal output will fail legitimately correct responses. A band calibrated to observed model behavior within a reasonable range is both stable and sensitive to regressions. Write the rubric based on what the model actually does, then tighten if the model improves.

Bot-block preflight before spending vision tokens

CONTEXT

Some URLs in eval fixtures return a Cloudflare CAPTCHA at the CDN edge before Playwright even loads the application. Running the full agent on a blocked URL costs tokens and produces a meaningless report.

DECISION

HTTP HEAD/GET check before each eval URL (`evals/run_evals.py:60`). Checks HTTP status code, content type, and text patterns (`"just a moment"`, `"captcha"`, `"access denied"`). Blocked URLs are marked `"skipped"`, not `"failed"`, so they don't contaminate the pass rate.

ALTERNATIVES CONSIDERED

Let the agent handle it and detect failure in the report; skip blocked URLs manually before each eval run; use a proxy to bypass CDN challenges.

TRADEOFFS ACCEPTED

Only catches CDN-edge challenges. JS-rendered CAPTCHAs served by the application after page load slip through to the agent run.

OUTCOME

Eval pass rates are not polluted by external infrastructure failures. The skipped count in the manifest distinguishes infrastructure problems from product regressions.

---

7 Security and Product Framing

`_sanitize_extracted.py` — indirect prompt injection defense

CONTEXT

Personas and friction strings extracted from agent output are fed back into downstream prompts — exec summary synthesis, multi-persona analysis, persona generation from the URL. A malicious website could embed instructions in on-page text that Claude captures and later re-injects into a downstream prompt.

DECISION

`_sanitize_extracted.py` caps persona strings at 200 characters, friction and recommendation fields at 500 characters, and strips strings matching patterns for role markers and instruction smuggles before any re-injection into a prompt.

ALTERNATIVES CONSIDERED

Trust model output; validate only at the final output boundary; use a dedicated guardrail model.

TRADEOFFS ACCEPTED

Overly aggressive sanitization could truncate legitimate UX findings on verbose pages.

OUTCOME

Indirect injection surface reduced. Added in Batch 27 after mapping the full data flow from extraction to re-injection. The character caps have not truncated meaningful findings in practice.

`TERMS.md` covering third-party TOS compliance and data handling

CONTEXT

Running automated browser sessions against third-party websites touches their terms of service. Without explicit terms of use, the tool's commercial viability is ambiguous and potential users have no documented basis for their own compliance decisions.

DECISION

`TERMS.md` added (Batch 46) covering: third-party TOS compliance (user's responsibility to verify), data retention (runs/ is gitignored and never transmitted), API cost model (user bears LLM costs), acceptable use, no-warranty disclaimer.

ALTERNATIVES CONSIDERED

README footnote; no terms; full legal terms drafted by counsel.

TRADEOFFS ACCEPTED

The TERMS.md provides clarity but not legal protection. It is not a substitute for counsel if the tool is used commercially at scale.

OUTCOME

Reduces ambiguity for potential users evaluating the tool. Required milestone before any public release.

`html.escape()` on all LLM-extracted fields in HTML report output

CONTEXT

`_build_html_report` and `_build_below_fold_html` build HTML via f-strings. All LLM-extracted strings — friction points, recommendations, observations, verdicts, first impressions, score notes, persona, below-fold findings — were interpolated directly without HTML escaping. `_sanitize_extracted.py` strips prompt-injection patterns (role markers, instruction smuggles) but does not HTML-escape.

THE GAP

A malicious site under audit could embed